Agent Roadmap v2: Flavor-Aware Healthier Recipe Substitution

Version 2 (corrected) | October 2026 | For the coding and analysis agent, two undergraduate students, and Prof. Ganesh Bagler (IIIT Delhi)

1. Mission brief (read this first)

Scope. You are the coding and analysis agent for a four-week research project: build and evaluate a flavor-aware healthier recipe substitution pipeline. Project 2 from the proposal (constraint-aware recipe generation, LLM prompting, LoRA fine-tuning) is out of scope. Do not build any of it.

One line. Given a recipe, replace 1 to 3 ingredients that contribute most to a nutrient we want to reduce (calories, saturated fat, sugar or sodium) with healthier alternatives, keep the dish plausible, and find which compatibility measure best identifies good swaps.

Who you work for. Two undergraduate students new to ML, supervised by Prof. Ganesh Bagler (IIIT Delhi; RecipeDB, FlavorDB, Foodoscope). The students must understand everything you build.

Research questions

1. Which measure best identifies an ingredient that fits a recipe: a popularity-only baseline, co-occurrence (PMI), flavor sharing, a learned embedding, or a combination?
2. Does FlavorDB flavor-molecule information improve on co-occurrence alone?
3. Does the answer differ across cuisines? The lab's 2015 study found negative food pairing in Indian regional cuisines (more shared flavor compounds, less co-occurrence). Never assume flavor sharing means compatibility; test it.
4. How much estimated nutrition improvement can swaps achieve while staying plausible?
5. Do the model's scores agree with human judgement of replacement pairs (Study 1, Section 7)?

Requirement from the professor. Evaluate the model by manually labelling replacement ingredient pairs (an original ingredient and its replacement) and comparing the model's scores with those human labels. This is Study 1 in Section 7 and is a MUST deliverable. If the professor meant something different, the students will ask him before Week 2 (suggested question at the end of Section 7).

How you work

- Explain in simple language. Define every ML term the first time, with a tiny example, in code comments or notebook text.
- Work in small steps. After each step print the key outputs and write a one-paragraph note on what they mean.
- Never invent numbers, column names or dataset facts. If unsure, inspect the data and say what you found.
- Always compare with simple baselines (including a popularity baseline), say when a difference is within noise, and report null or negative results.
- At the end of every week, stop at the gate, write a short status note with evidence in reports/, and wait for the students to confirm before starting the next week.

Priorities. Every task is marked MUST or IF TIME. Do all MUST items first. If the team falls behind, cut in this order: web app, Foodoscope baseline and expert rating, Type A pairs, LightGBM, sugar and sodium, per-cuisine tables, embeddings (last).

2. Ground rules, data and repository

2.1 Data sources

Source	Use	Status
RecipeDB	Recipes with ingredients parsed into name, quantity, unit, state; 29 ingredient categories; 5 dietary styles; estimated nutrition. The paper reports 118,171 recipes; the website overview gives different ingredient counts, so confirm which set we get	Access (bulk vs API, rate limits) to be confirmed with the professor
FlavorDB	Flavor molecules per ingredient	Mapping from RecipeDB ingredients to be confirmed
Foodoscope APIs	Substitution and compatibility output as baseline B6	Only if access is granted

Source	Use	Status
Food.com (earlier project)	Prototype compatibility measures only: no quantities, recipe-level nutrition only	Available now

Fallbacks. If RecipeDB access is slow, prototype the compatibility part on Food.com and keep every function source-agnostic. If nutrients per ingredient are sparse, restrict to the nutrients available and say so in the report.

2.2 Data caveats you must respect

- Food.com cannot support per-ingredient health calculations. Never use it for the health part.
- RecipeDB nutrition is estimated. Agreement between our recomputed nutrition and RecipeDB's values validates our pipeline against RecipeDB, not the true nutrition. Every improvement is an estimated improvement.
- Flavor molecules do not capture texture or cooking function (butter or sugar in baking). Start with function-safe swaps and document exclusions.
- Start with calories and saturated fat (MUST). Add sugar and sodium only if available per ingredient (IF TIME).
- Check whether RecipeDB provides servings. If not, report changes as the relative change in the whole-recipe total. Swaps are at equal weight, so the relative change is the same per serving.
- RecipeDB may already provide normalized ingredient names. Inspect them first and clean only what is needed.
- This is a computational study, not medical advice.

2.3 Definition of "healthier"

The targeted nutrient drops by at least 15% (relative), protein stays within 20% of the original, and the dietary style is preserved. These thresholds are placeholders until the professor confirms. Keep them in one config file.

2.4 Splits and leakage rules (non-negotiable)

1. Split recipes into train, validation and test (70/15/15) with a fixed seed. Never reuse a recipe across splits. Recipes with identical ingredient sets stay in the same split.

2. Split the training recipes again into a statistics set S (about 60%) and a model-training set M (about 40%).
3. All statistics come from S only: ingredient frequencies, PMI, co-occurrence counts, embeddings, vocabulary, the popularity baseline, and the cutoffs that define high or low PMI and flavor sharing.
4. The supervised compatibility model is fitted on M only, using features computed from S. This avoids the leak where a recipe's own ingredients inflate its features. At validation and test time, features also come from S.
5. Tune weights and hyperparameters on validation only. The test set is touched once, for final numbers.
6. Manual labels are used for evaluation only, never for tuning. Model scores for labelled pairs stay hidden until all labels are locked.
7. Negative examples for the supervised model are drawn matched on popularity (similar frequency in S) and, as a harder setting, from the same category. Report both. Purely random negatives are mostly rare ingredients and make the task too easy.

2.5 Statistics rules

- Use a paired bootstrap on the difference between two methods, on the same resamples, with at least 1,000 resamples. Resample recipes for recipe-level metrics and pairs for pair-level metrics.
- A difference is real only if the 95% interval of the difference excludes 0. Do not judge by whether the two methods' separate intervals overlap.
- Primary comparisons, fixed now to avoid fishing: (P1) PMI vs the popularity baseline on hide-and-recover MRR; (P2) the supervised model vs PMI alone; (P3) flavor sharing vs random on hide-and-recover MRR; (P4) the same methods on agreement with human labels. Everything else is exploratory and must be labelled as such. With many comparisons, interpret borderline results cautiously.

2.6 Repository layout and stack

```
recipe-swap/
  README.md        how to run everything, versions, seeds
  config.yaml      thresholds, seeds, paths, nutrient list
  data/raw/        never edited
  data/processed/  cleaned tables, mappings, splits (train/val/test, S/M)
  src/             parsing, grams, nutrition, measures, candidates, scoring,
eval
  notebooks/       one per week, plain-language explanations
  labels/          pair sheets, rater sheets, answer key (kept hidden)
  results/         tables, figures, bootstrap outputs
  reports/         weekly status notes, final report
```

Stack: Python, pandas, numpy, scikit-learn, matplotlib, gensim (Word2Vec) or a small PyTorch model for embeddings, LightGBM only if time. Record package versions in the README.

3. Week 1: data, cleaning, mappings, coverage

Goal. A parser and mapping layer the rest of the project can trust.

1. **(MUST) Data access and inspection.** Load whatever RecipeDB, FlavorDB and Food.com data exists. Report counts, columns, dtypes, missing rates, which nutrients exist per ingredient, whether servings exist, cuisine labels, and the ingredient name format. Do not assume column names; print them. Output: reports/week1_data_inventory.md.
2. **(MUST) Literature check.** Search for prior work on ingredient substitution, food pairing and healthier swaps (start with Shirai et al. 2021 and the lab's 2015 food-pairing paper). Output: a one-page note listing overlaps and how our comparison differs. Flag anything that threatens novelty.
3. **(MUST) Ingredient cleaning.** Reuse the earlier rules only where RecipeDB names need them: lowercase, singularize, strip prep words such as "diced", split compounds such as "salt and pepper", and handle ambiguous words ("clove" as spice vs garlic). Output: data/processed/ingredient_normalization.csv (raw name, clean name, rule applied).
4. **(MUST) Quantity to grams.** Parse quantity, unit and state, and convert to grams with a density or unit table. Log every ingredient that cannot be converted. Output: a grams column and an unconverted-ingredients report.

5. **(MUST) Nutrition and flavor mapping.** Map each clean ingredient to nutrition per 100 g (calories and saturated fat first) and to a FlavorDB entry. Output: ingredient_master.csv with category, nutrition per 100 g, FlavorDB id (or blank), mapping method and confidence.
6. **(MUST) Coverage report.** Share of ingredients, and of recipe weight, with both nutrition and flavor data, overall and per cuisine and category. Decide a minimum-coverage rule for keeping recipes.
7. **(MUST) Splits.** Create the train, validation and test split, then the S and M split of train, with fixed seeds. Group identical ingredient sets. Save recipe ids and check cuisine balance.
8. **(MUST) Hand-check.** Students and agent hand-check 50 random recipes for parsing and mapping errors and record the error rate.

Gate 1. The parser works on the hand-checked sample (target: at least 90% of ingredient lines correct, to be confirmed with the students) and coverage is acceptable. If not, fix before continuing, or switch the compatibility work to Food.com and say so.

Questions to raise with the professor this week: bulk data vs API and rate limits; nutrients available per ingredient; servings; the RecipeDB to FlavorDB mapping; the preferred definition of “healthier”; an expert who could rate pairs; and the meaning of the labelling requirement (Section 7).

4. Week 2: nutrition, candidates, compatibility measures, pair set

Goal. Every compatibility measure exists, is leakage-free and is tested objectively, and the Study 1 pair set is frozen and being labelled. Tracks A and B run in parallel.

Track A: health side (Person 1)

1. **(MUST) Nutrition per recipe** (per serving if servings exist). Quantity in grams times nutrition per gram, summed. Validate on a 200-recipe sample against RecipeDB’s listed values and report the median and 90th-percentile error.
2. **(MUST) High-impact ingredients.** For each recipe and target nutrient, rank ingredients by contribution. Report how concentrated the contribution is (share from the top 1, 2 and 3 ingredients).
3. **(MUST) Candidate generation.** For each target ingredient, list candidates from the same RecipeDB category plus a small curated function-safe set (for example fats and oils), with a lower value of the targeted nutrient per 100 g and a compatible dietary

style. Save the curated groups and exclusions in a reviewed file.

Track B: compatibility side (Person 2), all statistics from S only

4. **(MUST) Popularity baseline.** Frequency of each ingredient in S, overall and within category.
5. **(MUST) PMI (co-occurrence).** Recipe-level score of a candidate = mean (and also max) PMI with the other ingredients. Use a minimum pair count and smoothing for rare pairs and document the choice.
6. **(MUST) Flavor sharing.** Jaccard overlap of FlavorDB molecule sets between the candidate and the original, and between the candidate and each other ingredient. Report how many ingredients lack flavor data and how those cases are treated. Never silently score them as zero.
7. **(MUST) Learned embeddings.** Treat each recipe in S as a sentence of ingredients and train Word2Vec (or similar). Ingredient order in a recipe is arbitrary, so use a window large enough to cover the whole recipe. Fix the seed and report sensitivity to dimension and window.
8. **(MUST) Hide-and-recover.** In validation recipes hide one ingredient and rank candidates (all ingredients, then same-category only) with random, popularity, PMI, flavor sharing and embeddings. Report top-1, top-5, top-10 and MRR, overall and per cuisine. Keep cuisines with too few recipes out of the per-cuisine table and say so.

Track C: Study 1 pair set (both)

9. **(MUST) Build and freeze the pair set** following Section 7, as soon as candidates (step 3) and the PMI and Jaccard scores (steps 5 and 6) exist. Freeze it before the supervised model scores anything.
10. **(MUST) Independent labelling begins.** Each student labels alone. Finish by early Week 3.

Gate 2. Hide-and-recover runs end to end with all methods plus the random and popularity baselines, recomputed nutrition matches RecipeDB within the tolerance the students agree on, and the pair set is frozen. If nutrition does not match, find out why before building swaps on it.

5. Week 3: supervised model, scoring, swaps, baselines, label comparison

Goal. The full swap pipeline runs, is compared with baselines, and is checked against human judgement.

1. **(MUST) Supervised compatibility model (the core ML part).** Training labels come free from recipes: hide one ingredient, the true ingredient is a positive, and negatives are sampled as in rule 7 of Section 2.4. Features (from S): PMI, shared flavor molecules, embedding similarity, ingredient category, cuisine. Fit on M. Start with logistic regression. Evaluate on validation against PMI alone (P2) and inspect feature importance and per-cuisine behaviour to see where flavor sharing helps or hurts. LightGBM is IF TIME.
2. **(MUST) Lock labels and compare with the model** as set out in Section 7: weighted kappa, consensus labels, then the model-versus-human tables.
3. **(MUST) Scoring function.** Score = a x health gain + b x similarity to the original + c x compatibility with the remaining ingredients. Manual labels must not be used to choose the weights. Instead, sweep the trade-off between health gain and compatibility on validation recipes, measuring compatibility with the hide-and-recover score or the supervised model's probability, and show the resulting trade-off curve. The students and the professor choose the operating point.
4. **(MUST) Apply swaps.** Up to 3 swaps per recipe at equal weight, recompute nutrition, and record before and after.
5. **(MUST) Baselines for swaps.** B1 random same-category swap; B2 healthiest-only swap; B3 co-occurrence only; B4 flavor sharing only; B5 popularity-compatible swap (the most frequent same-category lower-nutrient candidate); B6 Foodoscope substitution (IF TIME, only if access exists).
6. **(MUST) Health and preservation outcomes.** Percent reduction in the targeted nutrient, share of swaps meeting the health criterion, protein side effects, same-category rate, and flavor-profile similarity before and after.
7. **(IF TIME) Cuisine-wise analysis** for cuisines with enough recipes.

Gate 3. Labels are locked, the model-versus-human comparison exists, baselines have run on the validation set, and the swap pipeline produces explained outputs for a sample of recipes. The test set is still untouched.

6. Week 4: final test run, report, demo

Goal. Final numbers, an honest write-up, and a demo the students can present. Keep the last two days as a buffer.

1. **(MUST) Freeze** weights, models, thresholds and the chosen operating point. Run everything once on the test set with the fixed seed. No further tuning.
2. **(MUST) Statistics.** Paired bootstrap 95% intervals (Section 2.5) for the primary comparisons, then the exploratory ones, marked as such. State clearly when a difference is within noise.
3. **(MUST) Cuisine-wise tables and null results.** Report where flavor information helps, where it hurts and where nothing differs. Include failure cases and unrealistic swaps.
4. **(MUST) Examples.** At least 10 transformed recipes with before and after ingredients, nutrition, compatibility scores and a plain explanation, including 2 or 3 bad swaps.
5. **(IF TIME) Study 2: rating of the final swaps.** See Section 7.5.
6. **(MUST) Report.** Introduction and related work (honest novelty statement), data and mapping with coverage, methods, Study 1, results, limitations and conclusion. Make clear which parts are ML and which are data engineering.
7. **(MUST) Demo notebook:** choose a recipe and target nutrient, and see ranked swaps with scores and nutrition before and after. A small web app is IF TIME.
8. **(MUST) Slides:** 10 to 12 slides following the report.
9. **(MUST) Reproducibility.** README with exact run order, seeds and package versions, and a fresh-environment test run of the main notebooks.

Gate 4 (final). All deliverables in Section 8 exist and the students can explain each result without the agent.

7. Study 1: manual labelling of replacement pairs (professor's requirement)

Why. Hide-and-recover only tells us whether a measure agrees with what recipes contain. It cannot tell us whether a replacement works to a human cook. Labelling replacement pairs by hand gives an independent check on every model, and is the evaluation the professor asked for.

7.1 What is labelled

- **Type B, replacement pairs (MUST, about 100 pairs).** An original ingredient, a candidate replacement, and the recipe context (title, cuisine and the other ingredients). Question: could the replacement take the place of the original in this dish without ruining it?
- **Type A, plain compatibility pairs (IF TIME, about 40 pairs).** Two ingredients, asked: would a cook plausibly use these two together in the same dish?
- **Sanity anchors.** Within the Type B set, about 10 obviously good and 10 obviously bad replacements.

7.2 Building the pool (agent)

- Contexts come from validation recipes. Originals are high-impact ingredients (Week 2, step 2) and replacements come from the candidate generator (Week 2, step 3), so the pool covers swaps the pipeline would really propose.
- Sample stratified into four cells: candidate-to-rest PMI (high or low) by candidate-to-original flavor Jaccard (high or low). Cutoffs are the medians computed from S before sampling. Take about 20 pairs per cell, spread across cuisines, categories and target nutrients.
- Use common ingredients only (a minimum frequency in S; choose and report the threshold), so the students can judge them.
- The agent may use scores to build the strata. The students never see scores or strata until all labels are locked.

7.3 Label scale

1 = does not work, 2 = possible or unusual, 3 = works well, and “?” when the rater does not know an ingredient. A free-text notes column is optional.

7.4 Sheet columns

Column	Content
pair_id	Shuffled id, no method name
type	A or B
ingredient_1, ingredient_2	Clean names (for B: original and replacement)

Column	Content
context	Recipe title, cuisine and other ingredients (type B); cuisine (type A)
rater1_label, rater2_label	Filled independently, hidden from each other
notes	Optional reason
consensus_label	Filled only after both are done

Model scores, strata and sources are kept in a separate answer-key file that the raters never see until all labels are locked.

7.5 Procedure

1. Freeze the pair list before labelling and before the supervised model scores it.
2. Each student labels alone, without searching for model output.
3. The agent computes weighted Cohen's kappa (ordinal). Record the first-round value. If it is low, tighten the written guidelines and discuss, but still report the first-round value.
4. Resolve disagreements by discussion into a consensus label. Drop "?" pairs and count them.
5. **Expert (IF TIME).** If the professor suggests an expert, they rate a subset of 30 to 40 pairs, and agreement with the students is reported.
6. **Study 2 (IF TIME).** Rate 30 to 40 swaps produced by the final pipeline and the baselines, blind and shuffled, with the same scale. Compare methods by mean human rating with paired bootstrap intervals.

7.6 Scores to compare

Score every pair with these, all computed from S only: random; popularity (log frequency of the replacement); PMI (mean and max with the rest of the recipe); flavor Jaccard (replacement vs original, and vs the rest); embedding cosine; the supervised model; and a combined plausibility score (similarity plus compatibility, without the health term).

7.7 Metrics

- Spearman correlation between score and consensus label, using all labels.

- AUROC for separating label 3 from label 1. Label 2 is excluded here; report how many pairs remain.
- Precision in the top 20% of scores, defined as the share of label-3 pairs among them.
- Each with paired bootstrap 95% intervals over pairs, including differences between methods (P4).
- Breakdowns by the four strata, by cuisine (if enough pairs) and by type.
- Report which measure agrees with the humans when PMI and flavor sharing disagree.
- Compare with the hide-and-recover ranking: do the measures rank in the same order on both tests? If not, say so and discuss.
- Show the model's confident mistakes (high score with label 1, and the reverse) as examples.

7.8 Limits to state in the report

Two students are not expert cooks, so labels reflect their cultural familiarity. About 100 pairs gives wide confidence intervals. A pair that is rare in recipes may still be a good pair. Labels judge plausibility, not health. Do not oversell results.

7.9 Question for the professor

"By manually labelling the replacement ingredient pairs and comparing with the model, do you mean that we label original-to-replacement swaps (with the recipe context) as plausible or not, and compare those labels with the model's scores?"

8. Baselines, risks and deliverables

8.1 Baselines

Hide-and-recover and pair scoring: random, popularity, PMI, flavor sharing, embedding. Swaps: B1 random same-category, B2 healthiest-only, B3 co-occurrence only, B4 flavor sharing only, B5 popularity-compatible, B6 Foodoscope (IF TIME).

8.2 Risks and what the agent should do

Risk	Response
RecipeDB or FlavorDB access delayed	Prototype compatibility on Food.com; keep code source-agnostic; tell the students immediately

Risk	Response
Nutrients per ingredient sparse	Restrict to the nutrients available and state it
Flavor data does not help	Report it as a finding; co-occurrence stays the main measure
Popularity shortcut inflates results	Popularity baseline in every comparison
Swaps look unrealistic	Function-safe categories only; analyze failures with the Study 1 labels (never tune on them)
Mapping errors	Hand-check samples; log the confidence of each mapping
Results within noise	Say so with the paired interval; do not rank methods whose difference includes 0
Overlap with existing work	Literature note in Week 1; no novelty claims beyond a controlled per-cuisine comparison
Schedule overrun	Use the cut order in Section 1

8.3 Stop and ask the students when

A threshold or definition must be chosen (for example “healthier”), a dataset fact is uncertain, a result looks too good (check for leakage first), a gate fails, or you want to deviate from this roadmap.

8.4 Deliverables checklist

- Reproducible code and notebooks, one per week, with plain-language explanations.
- Ingredient normalization table and nutrition/flavor mapping with a coverage report.
- Hide-and-recover table with paired confidence intervals, including the popularity baseline, overall and per cuisine.
- Study 1: pair labels, rater agreement, and the model-versus-human comparison tables.
- Baseline comparison (B1 to B5, B6 if available) with health and preservation outcomes.
- At least 10 transformed recipes with explanations.
- Final report, slides and a short demo.
- README with seeds, versions and run order.

9. References

1. Jain A, Rakhi NK, Bagler G. Analysis of Food Pairing in Regional Cuisines of India. PLOS ONE 10(10): e0139539 (2015). https://doi.org/10.1371/journal.pone.0139539
2. Shirai SS, Seneviratne O, Gordon ME, Chen C-H, McGuinness DL. Identifying Ingredient Substitutions Using a Knowledge Graph of Food. Frontiers in Artificial Intelligence 3:621766 (2021). https://doi.org/10.3389/frai.2020.621766
3. RecipeDB: a resource for exploring recipes. Database (Oxford), 2020, baaa077. https://doi.org/10.1093/database/baaa077